

TicketOutTheDoor2.R

gcrone14

2026-10-05

```
#### Ticket out-the-door (2) - Answer ####
```

Instructions: Import the starbucks data and clean it up by removing any items for which the grams of fat or carbs are missing. Feel free to also select which variables are of most interest.

Once you do so, pipe in the following lines of code to create a new variable, called category, which gives more detailed info regarding what an item includes. (You'll need to un-comment it out)

```
# YOUR_STARBUCKS_DATA |>
# mutate(category = case_when(
#   str_detect(item, "Frappuccino") ~ "Frappuccino",
#   str_detect(item, "Latte/Cappuccino/Macchiato/Mocha/Americano") ~ "Espresso drink",
#   str_detect(item, "Coffee/Cold Brew/Roast") ~ "Coffee",
#   str_detect(item, "Tea/Tazo/Teavana") ~ "Tea",
#   str_detect(item, "Refresher/Pink Drink/Violet Drink") ~ "Refreshers",
#   str_detect(item, "Sandwich/Panini/Wrap") ~ "Sandwich/wrap",
#   str_detect(item, "Salad") ~ "Salad",
#   str_detect(item, "Protein Box/Protein Bowl") ~ "Protein meal",
#   str_detect(item, "Muffin/Cookie/Croissant/Scone/Cake/Brownie/Doughnut/Loaf/Tart") ~ "Bakery",
#   TRUE ~ NA
# ))
```

After making this modification, remove any data that's missing a category (i.e., where category is NA). Now, save that result into a new object, and create a plot (or plots) to answer the following research question: Does the relationship between carbs and fat differ based on the CATEGORY of each item?

Make sure that the plot(s) is (are) not too "busy" with regard to the category variable (facet_wrap() is your friend!).

```
# Loading packages
library(here)
```

```
## here() starts at /Users/gcrone14/Desktop/R Projects/PSYC3031-F26
```

```
library(psych)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2     4.0.3      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr       1.2.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x ggplot2::%+%( ) masks psych::%+%( )
## x ggplot2::alpha() masks psych::alpha()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

# Loading the data
star_items <- read.csv(here("Data/starbucks_items.csv")) |>
  # For simplicity of viewing
  as_tibble()
```

Here we do a simple data cleaning/modification. Note that this step is technically optional in this case. It is nice to clean the data up a tad.

```
star_items_final <- star_items |>
  # You may not wish to select only these variables,
  # but it's up to you.
  select(item, type, fat_g, carb_g) |>
  # We filter out missing data because it doesn't
  # provide any useful information.
  filter(!is.na(fat_g) & !is.na(carb_g))

# Add in new variable based on code above
star_items_full <- star_items_final |>
  mutate(category = case_when(
    str_detect(item, "Frappuccino") ~ "Frappuccino",
    str_detect(item, "Latte|Cappuccino|Macchiato|Mocha|Americano") ~ "Espresso drink",
    str_detect(item, "Coffee|Cold Brew|Roast") ~ "Coffee",
    str_detect(item, "Tea|Tazo|Teavana") ~ "Tea",
    str_detect(item, "Refresher|Pink Drink|Violet Drink") ~ "Refreshers",
    str_detect(item, "Sandwich|Panini|Wrap") ~ "Sandwich/wrap",
    str_detect(item, "Salad") ~ "Salad",
    str_detect(item, "Protein Box|Protein Bowl") ~ "Protein meal",
    str_detect(item, "Muffin|Cookie|Croissant|Scone|Cake|Brownie|Doughnut|Loaf|Tart") ~ "Bakery",
    TRUE ~ NA
  )) |>
  filter(!is.na(category))
```

Let's get a better understanding of the breakdown of various category items

```
star_items_full |> count(category, sort = TRUE)
```

```
## # A tibble: 9 x 2
##   category      n
##   <chr>      <int>
## 1 Tea        34
## 2 Bakery     32
## 3 Coffee     24
## 4 Espresso drink 23
## 5 Sandwich/wrap 20
## 6 Protein meal   8
```

```
## 7 Frappuccino      5
## 8 Refreshers       5
## 9 Salad            5
```

The labels suggest that most items (without missing data) are tea ($N = 34$), bakery items ($N = 32$), and coffee ($N = 24$), whereas frappuccinos ($N = 5$), refreshers ($N = 5$), and salads ($N = 5$) were the least popular.

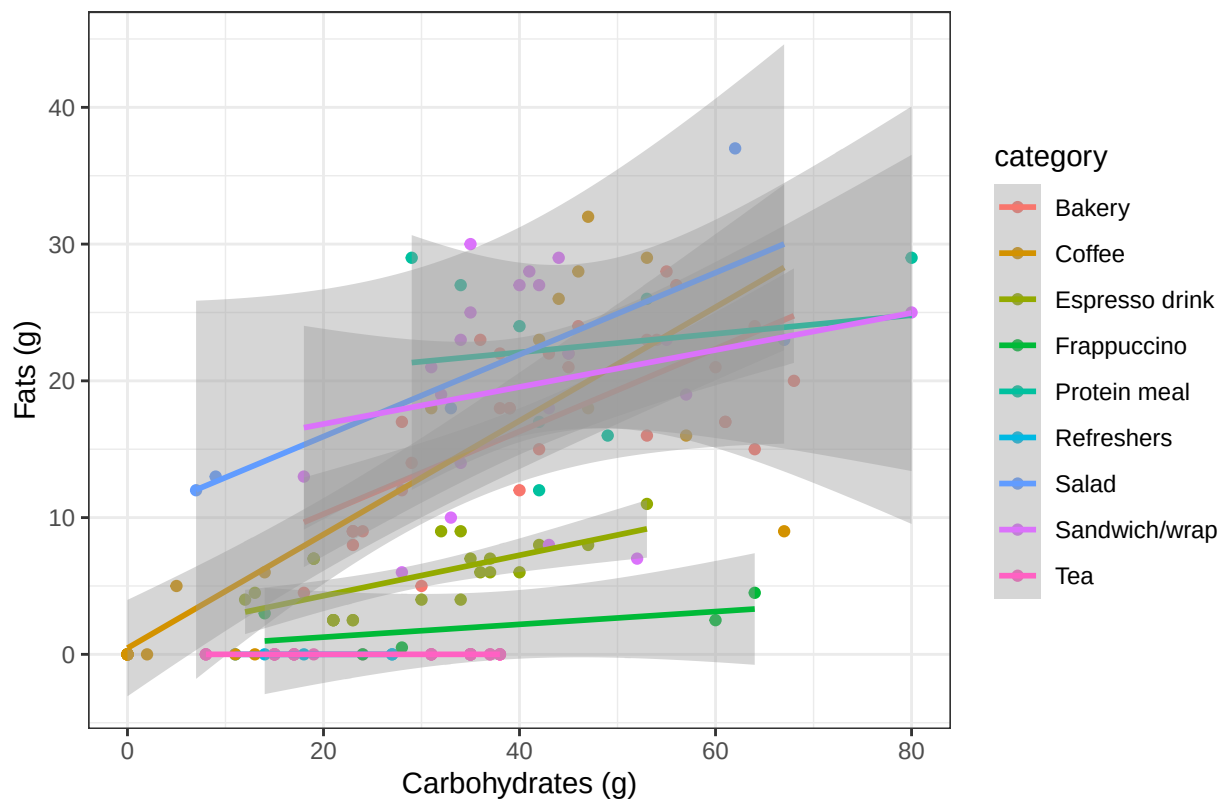
(SIDENOTE: Frappuccinos were likely not here often because their data were missing regarding their fat and sugar content.)

Now let's focus in on our research question: how does the relationship between carbs and fat differ based on the category of item?

The plot we did last time for our answer works particularly well. We tweak it by swapping out the variable type for category.

```
star_items_full |>
  ggplot(aes(x = carb_g, y = fat_g, color = category)) +
  geom_point() +
  # Good addition: Fits a line of best fit with margin of error
  geom_smooth(method = "lm", formula = "y ~ x") +
  # Good addition: Adds axis labels
  labs(x = "Carbohydrates (g)", y = "Fats (g)",
       title = "Figure 1") +
  # Good addition: Makes theme simpler
  theme_bw()
```

Figure 1

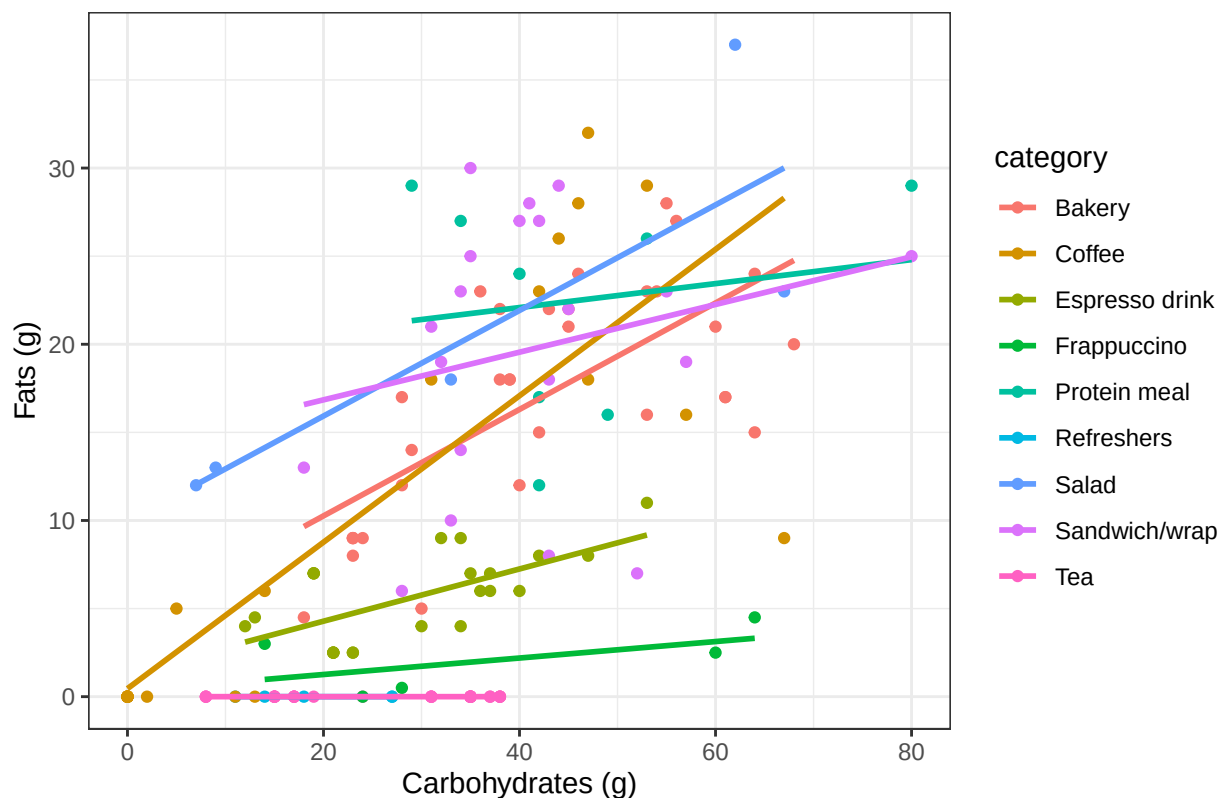


This plot is ok, but is far too busy to make out clear patterns.

Perhaps if we omit the se widths on `geom_smooth()`, it'll look better?

```
star_items_full |>
  ggplot(aes(x = carb_g, y = fat_g, color = category)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y ~ x",
             # Remove SE widths on bars
             se = FALSE) +
  labs(x = "Carbohydrates (g)", y = "Fats (g)",
       title = "Figure 1") +
  theme_bw()
```

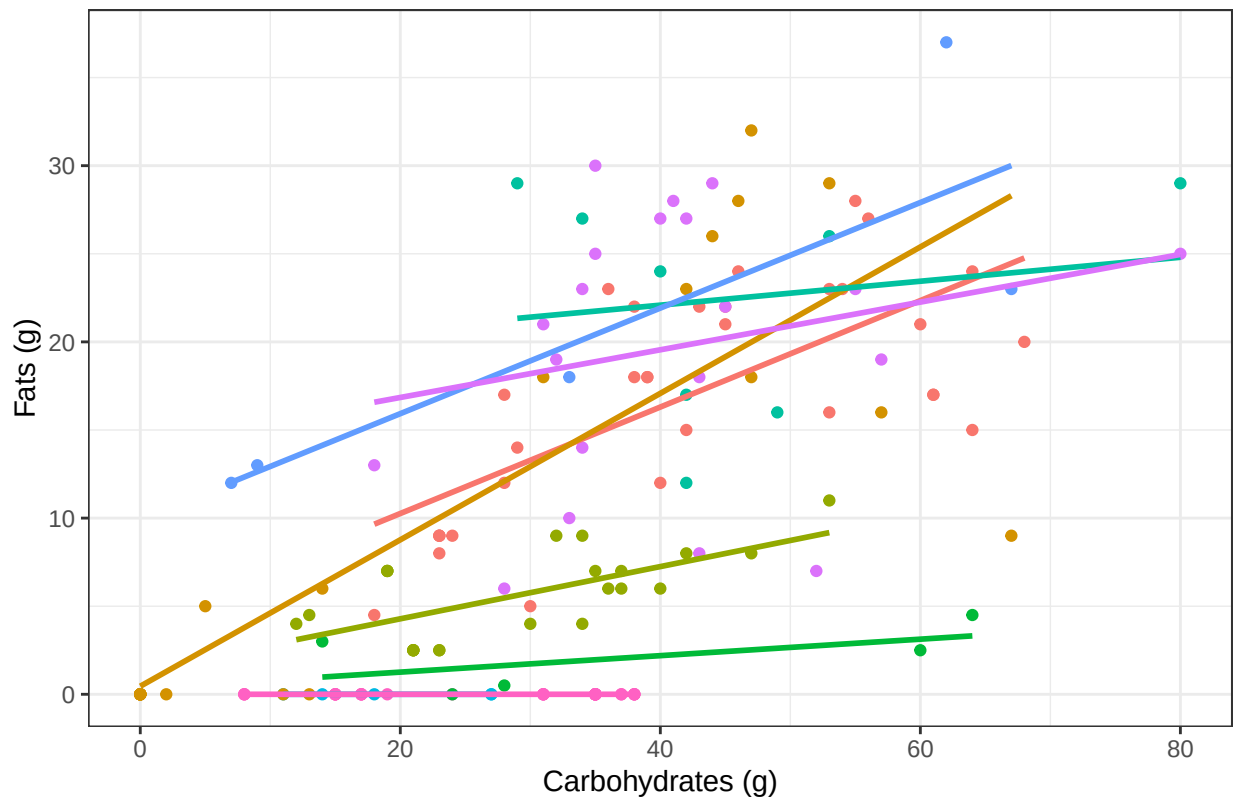
Figure 1



It looks a tad better, but is still very busy and hard to decipher clear patterns. Let's omit the plot legend and see if that helps.

```
star_items_full |>
  ggplot(aes(x = carb_g, y = fat_g, color = category)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y ~ x",
             se = FALSE) +
  labs(x = "Carbohydrates (g)", y = "Fats (g)",
       title = "Figure 1") +
  theme_bw() +
  # Add theme to get rid of legend
  theme(legend.position = "none")
```

Figure 1

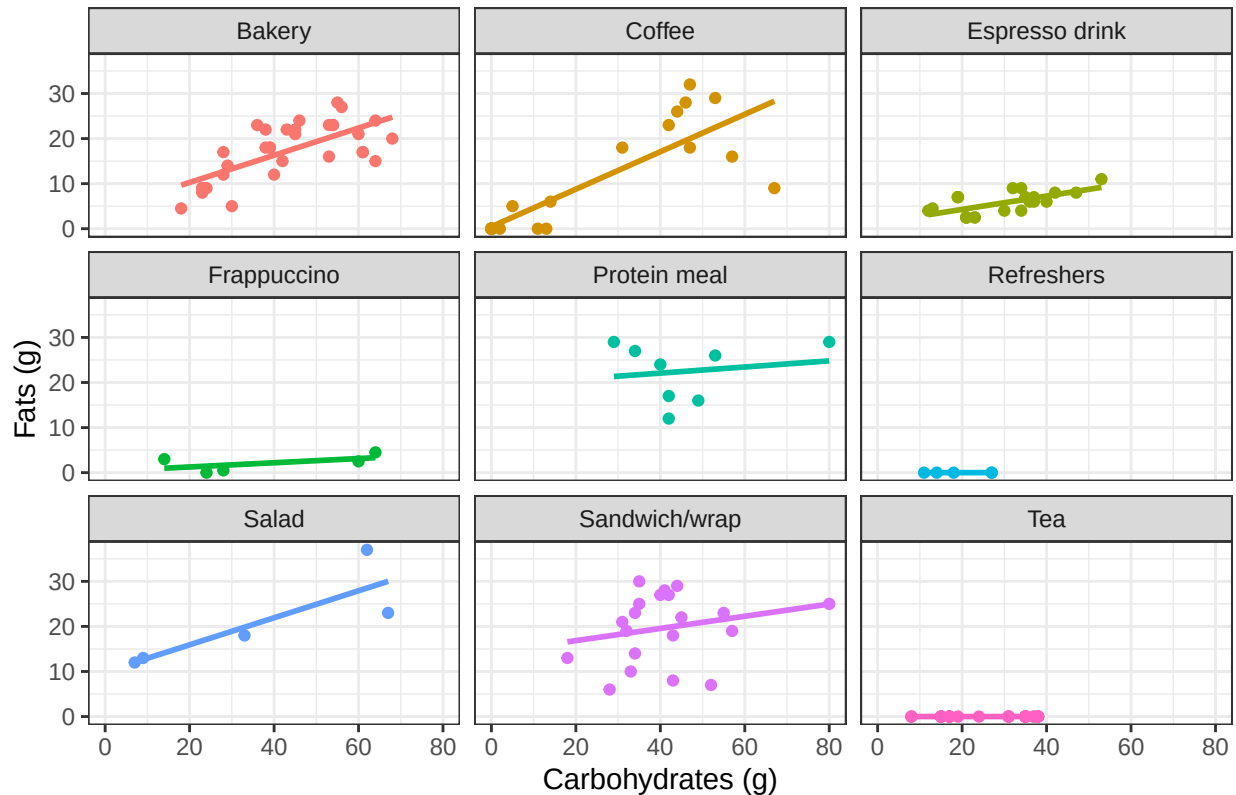


A bit better, but now we have another problem: what does each color mean?

I think the most reasonable solution with so much data is to use `facet_wrap()`. Doing so enables us to split up the plot by the category of item, making it easier to see what's going on.

```
star_items_full |>
  ggplot(aes(x = carb_g, y = fat_g, color = category)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y ~ x",
             se = FALSE) +
  labs(x = "Carbohydrates (g)", y = "Fats (g)",
       title = "Figure 1") +
  theme_bw() +
  theme(legend.position = "none") +
  # Add facet wrap
  facet_wrap(~category)
```

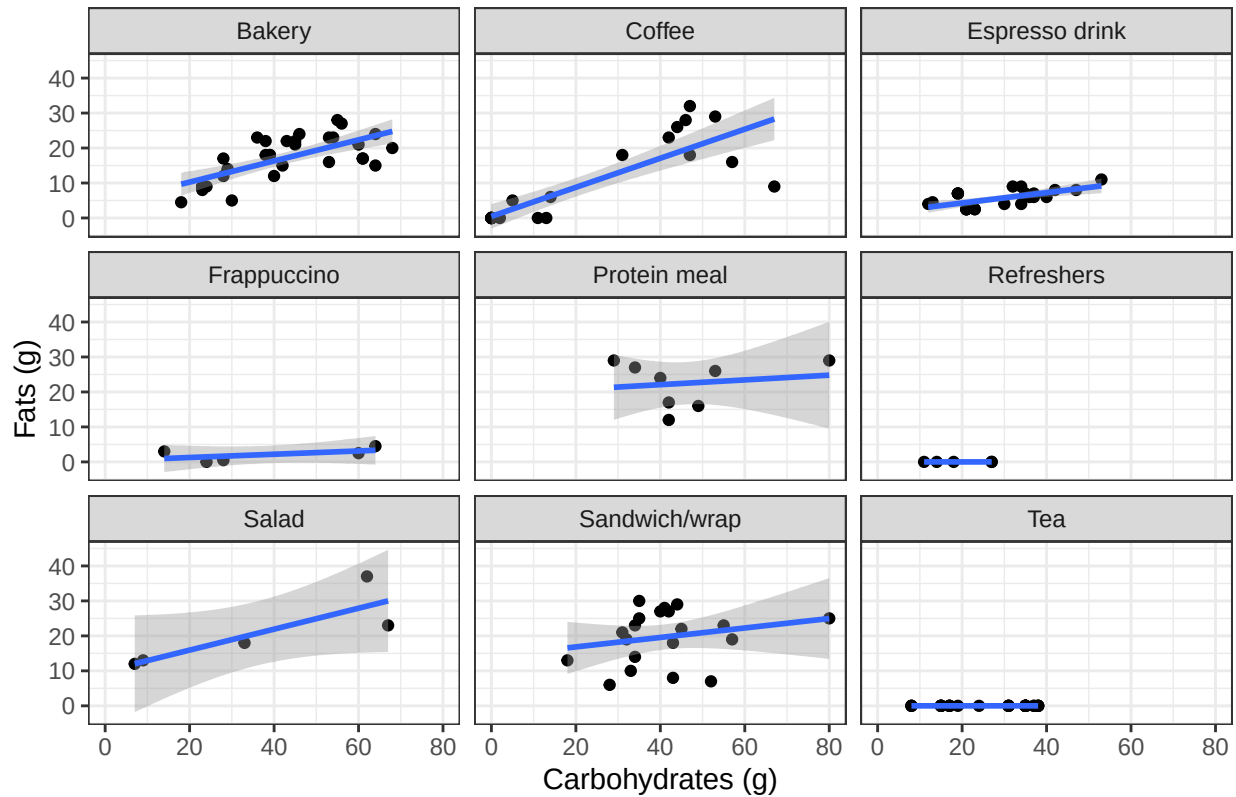
Figure 1



This is dramatically better! Colors no longer matter all that much, since each group is clearly separated out. We'll also omit the `legend.position = "none"` (because we'll no longer need a legend), and allow the error bars to exist.

```
star_items_full |>
  # Here, we omit the color argument
  ggplot(aes(x = carb_g, y = fat_g)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y ~ x") +
  labs(x = "Carbohydrates (g)", y = "Fats (g)",
       title = "Figure 1") +
  theme_bw() +
  # (We omit the legend.position = none, since there
  # is no longer a legend)
  facet_wrap(~category)
```

Figure 1



This graph, I think, works well because it expediently answer the question we cared about: does the category of item affect the relationship between fat and carbs?

The answer is a resounding yes: The relationship is strong and positive within the bakery and coffee groups, and is far less strong (weak to almost non-existent) within all other groups.

This plot also shows you the raw data points; more data makes the relationships shown have more evidence. (Less data means they're less reliable, hence the wider error bars.)

It's nice because we can clearly see that no tea or refreshers have fat, which is why the relationship doesn't hold within that group.

This is a good example of what's called a MODERATION: the item category moderates the relationship between carbs and fat.